



Vehículos Aeroespaciales: Misiles Prácticas - Sesión 1

Objetivos

1. Introducción a Matlab.
2. Modelo unidimensional: Familiarización con las ecuaciones básicas y los integradores de Matlab.
3. Integración y comparación con la solución analítica en los casos más sencillos.
4. Evaluación del efecto de la atenuación y variación de la gravedad.

Referencias

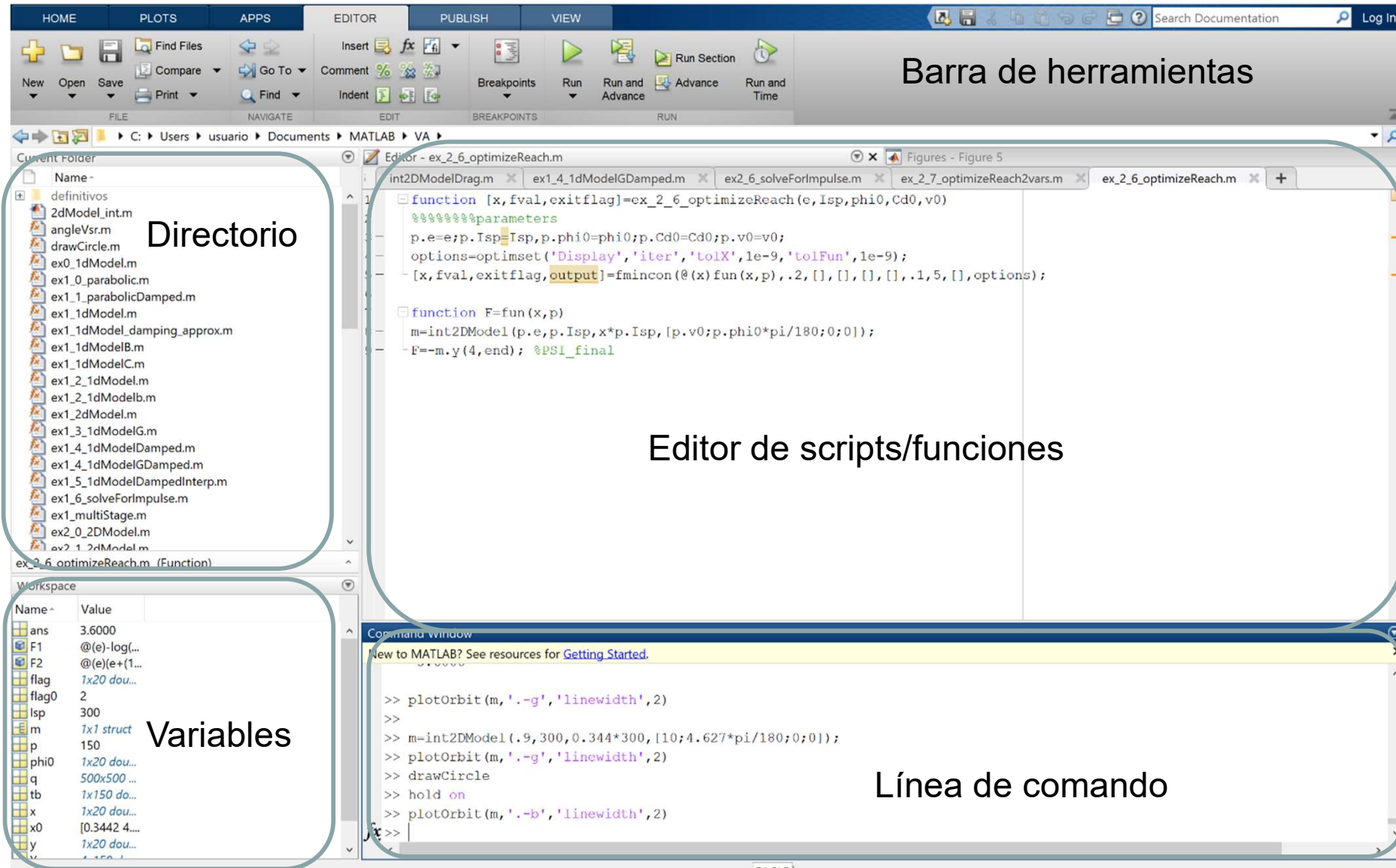
- Guiones de Vehículos Espaciales y Misiles.
- <http://web.mit.edu/18.06/www/Spring09/matlab-cheatsheet.pdf>

Introducción a Matlab

- Descrito por Mathworks como plataforma optimizada para la resolución de problemas científicos de ingeniería.
- La plataforma incluye un entorno de programación (con un lenguaje propio), herramientas de visualización y una amplia cantidad de funciones pre-programadas, agrupadas en librerías especializadas (*toolbox*). Permite la ejecución de código desde funciones guardadas por el usuario, o en tiempo real desde línea de comando.
- Rápida curva de aprendizaje, intuitivo.
- Extendido, y con una gran comunidad de usuarios.

Se cubrirá la funcionalidad **básica**, aplicada a la resolución numérica de ecuaciones diferenciales y al análisis de sus soluciones.

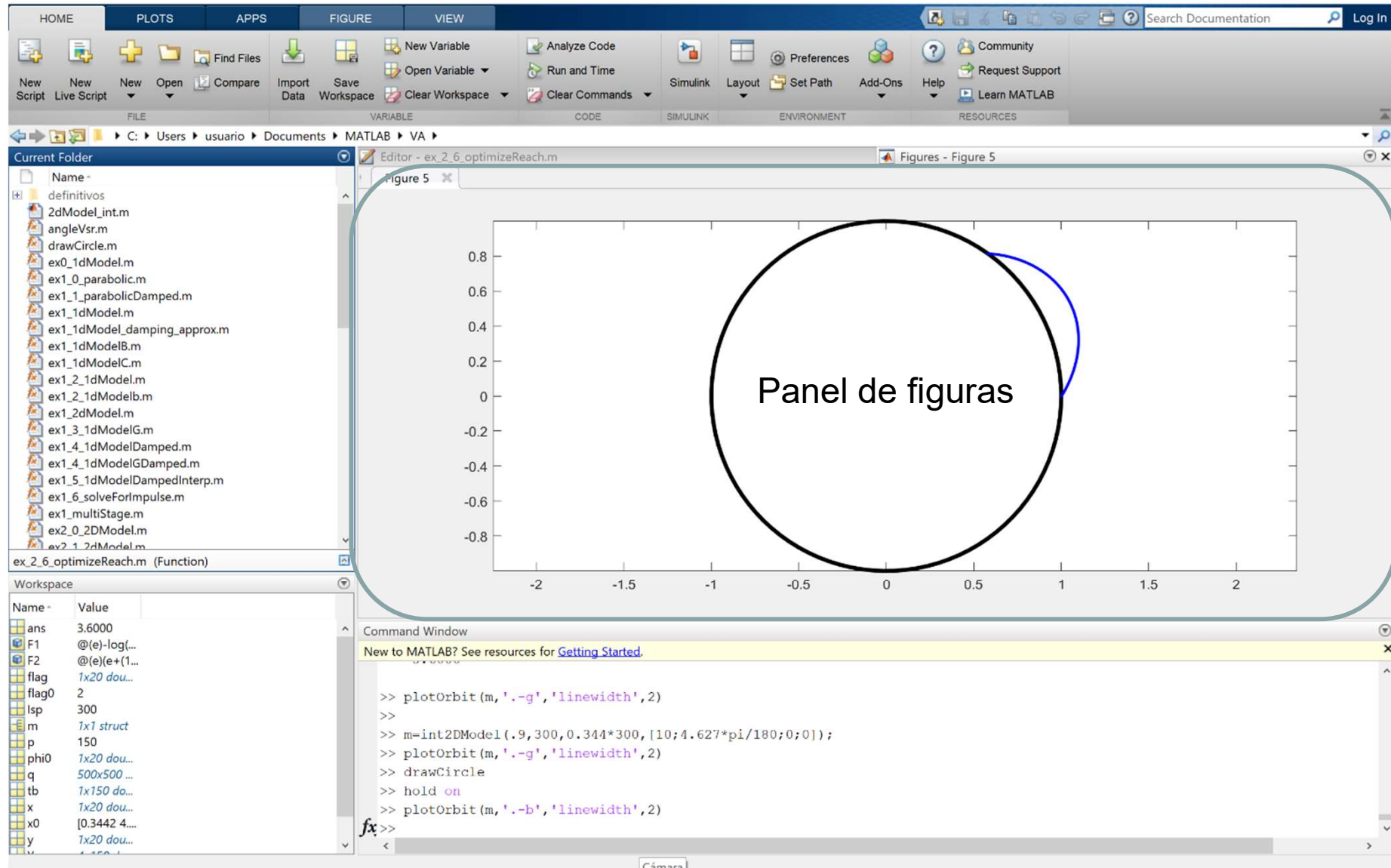
Entorno



The screenshot shows the MATLAB R2019b desktop environment. The interface is divided into several panes:

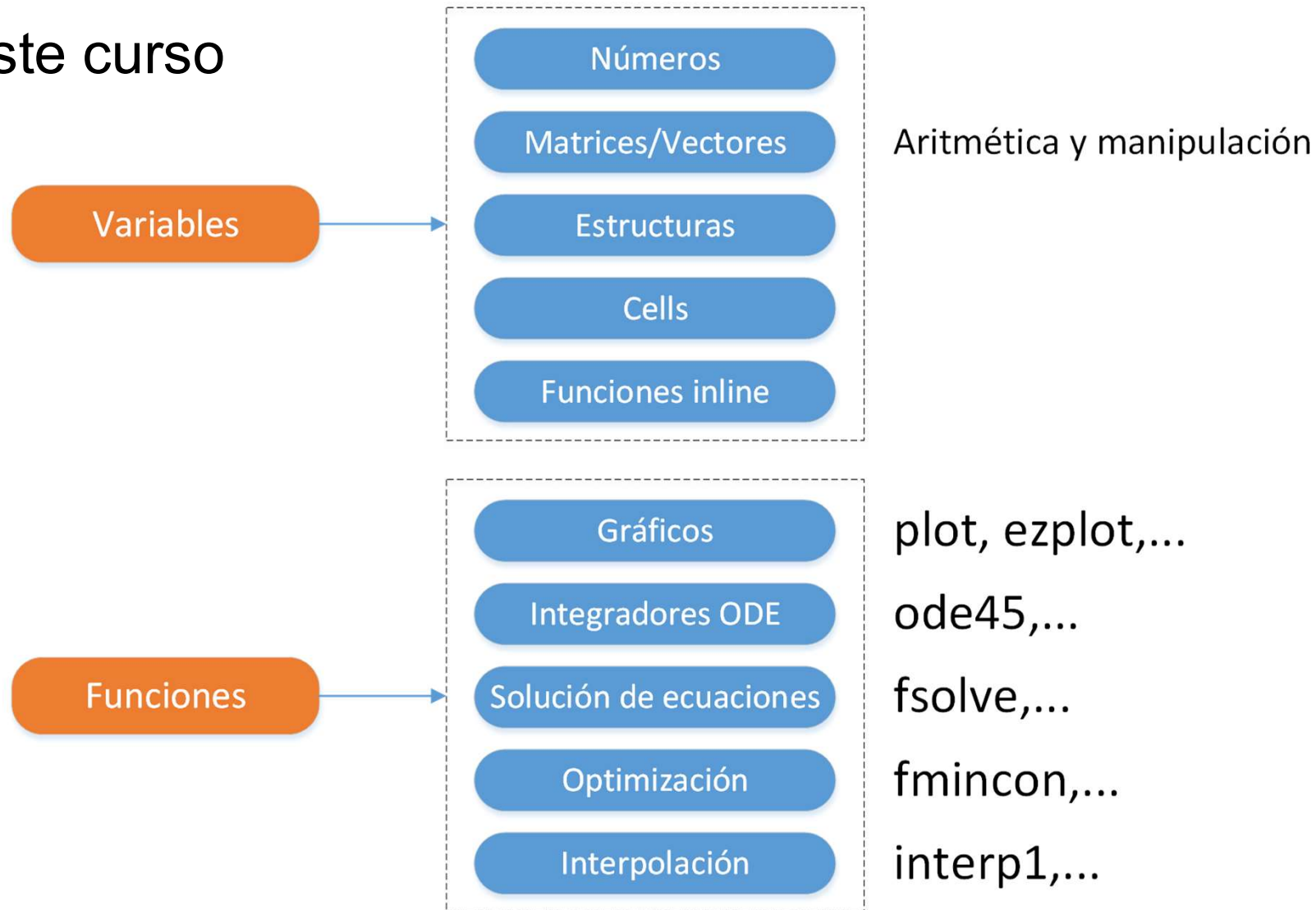
- Barra de herramientas (Toolbar):** Located at the top, it contains icons for file operations (New, Open, Save, Print), navigation (Go To, Find), editing (Insert, Comment, Indent), breakpoints, and running code (Run, Run and Advance, Run Section, Run and Time).
- Directorio (Current Folder):** Located on the left, it displays a list of files and folders in the current directory, including 'definitivos', '2dModel_int.m', 'angleVsr.m', 'drawCircle.m', and various 'ex' files.
- Editor de scripts/funciones (Editor):** The central pane shows the code editor for 'ex_2_6_optimizeReach.m'. It contains MATLAB code for a function that optimizes a reach problem using 'fmincon'.
- Variables (Workspace):** Located at the bottom left, it shows a table of variables in the workspace, including 'ans', 'F1', 'F2', 'flag', 'flag0', 'lsp', 'm', 'p', 'phi0', 'q', 'tb', 'x', 'x0', 'y', and 'z'.
- Línea de comando (Command Window):** Located at the bottom right, it shows the command prompt with several commands entered, such as 'plotOrbit(m, \'.-g\', 'linewidth', 2)', 'm=int2DModel(.9, 300, 0.344*300, [10; 4.627*pi/180; 0; 0])', 'plotOrbit(m, \'.-g\', 'linewidth', 2)', 'drawCircle', 'hold on', and 'plotOrbit(m, \'.-b\', 'linewidth', 2)'.

Entorno



Matlab Core

En este curso



*usar los comandos doc y help seguidos del nombre de la función para consultar la ayuda.

Cheatsheet

A Matlab Cheat-sheet (MIT 18.06, Fall 2007)

Basics:

save 'file.mat' save variables to *file.mat*
load 'file.mat' load variables from *file.mat*
diary on record input/output to file *diary*
diary off stop recording
whos list all variables currently defined
clear delete/undefine all variables
help command quick help on a given *command*
doc command extensive help on a given *command*

Defining/changing variables:

x = 3 define variable *x* to be 3
x = [1 2 3] set *x* to the 1×3 row-vector (1,2,3)
x = [1 2 3]; same, but don't echo *x* to output
x = [1;2;3] set *x* to the 3×1 column-vector (1,2,3)
A = [1 2 3 4; 5 6 7 8; 9 10 11 12];
set *A* to the 3×4 matrix with rows 1,2,3,4 etc.
x(2) = 7 change *x* from (1,2,3) to (1,7,3)
A(2,1) = 0 change *A*_{2,1} from 5 to 0

Arithmetic and functions of numbers:

3*4, 7+4, 2-6 8/3 multiply, add, subtract, and divide numbers
3^7, 3^(8+2i) compute 3 to the 7th power, or 3 to the 8+2i power
sqrt(-5) compute the square root of -5
exp(12) compute e^{12}
log(3), log10(100) compute the natural log (ln) and base-10 log ($\log_{10}$)
abs(-5) compute the absolute value |-5|
sin(5*pi/3) compute the sine of 5π/3
besselj(2,6) compute the Bessel function $J_2(6)$

Arithmetic and functions of vectors and matrices:

x * 3 multiply every element of *x* by 3
x + 2 add 2 to every element of *x*
x + y element-wise addition of two vectors *x* and *y*
A * y product of a matrix *A* and a vector *y*
A * B product of two matrices *A* and *B*
x * y not allowed if *x* and *y* are two column vectors!
x .* y element-wise product of vectors *x* and *y*
A^3 the square matrix *A* to the 3rd power
x^3 not allowed if *x* is not a square matrix!
x.^3 every element of *x* is taken to the 3rd power
cos(x) the cosine of every element of *x*
abs(A) the absolute value of every element of *A*
exp(A) e to the power of every element of *A*
sqrt(A) the square root of every element of *A*
expm(A) the matrix exponential e^A
sqrtm(A) the matrix whose square is *A*

Transposes and dot products:

x.', A.' the transposes of *x* and *A*
x', A' the complex-conjugate of the transposes of *x* and *A*
x' * y the dot (inner) product of two column vectors *x* and *y*
dot(x,y), sum(x.*y) ...two other ways to write the dot product
x * y' the outer product of two column vectors *x* and *y*

Constructing a few simple matrices:

rand(12,4) a 12×4 matrix with uniform random numbers in [0,1)
randn(12,4) a 12×4 matrix with Gaussian random (center 0, variance 1)
zeros(12,4) a 12×4 matrix of zeros
ones(12,4) a 12×4 matrix of ones
eye(5) a 5×5 identity matrix *I* ("eye")
eye(12,4) a 12×4 matrix whose first 4 rows are the 4×4 identity
linspace(1.2,4.7,100) row vector of 100 equally-spaced numbers from 1.2 to 4.7
7:15 row vector of 7,8,9,...,14,15
diag(x) matrix whose diagonal is the entries of *x* (and other elements = 0)

Portions of matrices and vectors:

x(2:12) the 2nd to the 12th elements of *x*
x(2:end) the 2nd to the last elements of *x*
x(1:3:end) every third element of *x*, from 1st to the last
x(:) all the elements of *x*
A(5,:) the row vector of every element in the 5th row of *A*
A(5,1:3) the row vector of the first 3 elements in the 5th row of *A*
A(:,2) the column vector of every element in the 2nd column of *A*
diag(A) column vector of the diagonal elements of *A*

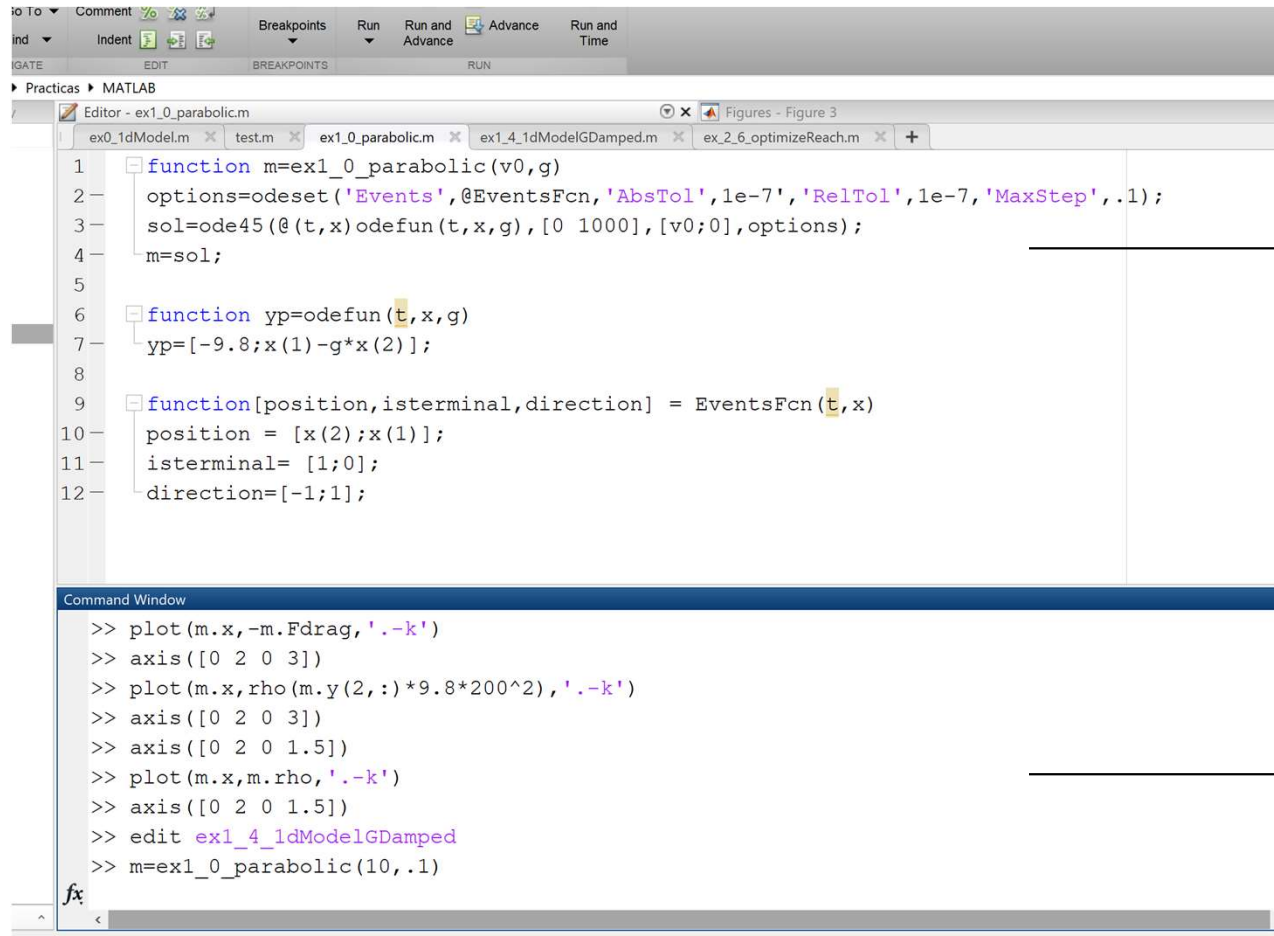
Solving linear equations:

A \ b for *A* a matrix and *b* a column vector, the solution *x* to $Ax=b$
inv(A) the inverse matrix A^{-1}
[L,U,P] = lu(A) the LU factorization $PA=LU$
eig(A) the eigenvalues of *A*
[V,D] = eig(A) the columns of *V* are the eigenvectors of *A*, and the diagonals diag(D) are the eigenvalues of *A*

Plotting:

plot(y) plot *y* as the *y* axis, with 1,2,3,... as the *x* axis
plot(x,y) plot *y* versus *x* (must have same length)
plot(x,A) plot columns of *A* versus *x* (must have same # rows)
loglog(x,y) plot *y* versus *x* on a log-log scale
semilogx(x,y) plot *y* versus *x* with *x* on a log scale
semilogy(x,y) plot *y* versus *x* with *y* on a log scale
fplot(@(x) ...expression..., [a,b]) plot some expression in *x* from *x=a* to *x=b*
axis equal force the *x* and *y* axes of the current plot to be scaled equally
title('A Title') add a title *A Title* at the top of the plot
xlabel('blah') label the *x* axis as *blah*
ylabel('blah') label the *y* axis as *blah*
legend('foo', 'bar') label 2 curves in the plot *foo* and *bar*
grid include a grid in the plot
figure open up a new figure window

Ejecución de código



The image shows the MATLAB environment. The Editor window displays a script named `ex1_0_parabolic.m` with the following code:

```

1 function m=ex1_0_parabolic(v0,g)
2     options=odeset('Events',@EventsFcn,'AbsTol',1e-7,'RelTol',1e-7,'MaxStep',.1);
3     sol=ode45(@(t,x)odefun(t,x,g),[0 1000],[v0;0],options);
4     m=sol;
5
6 function yp=odefun(t,x,g)
7     yp=[-9.8;x(1)-g*x(2)];
8
9 function[position,isterminal,direction] = EventsFcn(t,x)
10    position = [x(2);x(1)];
11    isterminal= [1;0];
12    direction=[-1;1];
  
```

The Command Window shows the following commands being executed:

```

>> plot(m.x,-m.Fdrag,'.-k')
>> axis([0 2 0 3])
>> plot(m.x,rho(m.y(2,:)*9.8*200^2),'.-k')
>> axis([0 2 0 3])
>> axis([0 2 0 1.5])
>> plot(m.x,m.rho,'.-k')
>> axis([0 2 0 1.5])
>> edit ex1_4_1dModelGDamped
>> m=ex1_0_parabolic(10,.1)
  
```

En archivo .m

- Scripts y funciones
- Se invoca desde línea de comando*

En línea de comando

- Variables en workspace

*Para que un script o función pueda ser invocado desde línea de comando, primero hay que guardarlo

Manipulación de matrices

Acceso a matrices

```
>> B = [1 2 3; 4 5 6; 7 8 9]
B =
1 2 3
4 5 6
7 8 9
```

```
>> B(:,1)
ans =
1
4
7
```

```
>> B(1,:)
ans =
1 2 3
```

```
>> B(1:2,1)
ans =
1
4
```

```
>> B(:)
ans =
1
4
7
2
5
8
3
6
9
```

Creación de vectores

```
>> t = [0:0.1:10]
t =
0 0.1000 0.2000 0.3000 0.4000 0.5000 0.6000 0.7000 0.8000 0.9000 1.0000
```

```
>> t=linspace(0,1,10)
t =
0 0.1111 0.2222 0.3333 0.4444 0.5556 0.6667 0.7778 0.8889 1.0000
```

```
>> length(t)
ans =
10
```

```
>> size(t)
ans =
1 10
```

```
>> size(t,2)
ans =
10
```

Operaciones con matrices y vectores

$x * 3$ multiplicación de todos los elementos de x por 3
 $x + 2$ suma 2 a todo element de x
 $x + y$ suma element a element de los vectores x e y
 $A * y$ producto de la matriz A y el vector y
 $A * B$ product matricial de A y B
 $x .* y$ producto **elemento a elemento** de x e y
 x^3 no permitido si x no es una matriz cuadrada
 $x.^3$ aplicado **element a elemento**

```
>> A=[1 0 0;0 1 0;0 0 1];B =[1 2 3; 4 5 6; 7 8 9];x=[1 2 3];y=[1 1 1];
```

```
>> A*B
```

```
ans =
     1     2     3
     4     5     6
     7     8     9
```

```
>> A.*B
```

```
ans =
     1     0     0
     0     5     0
     0     0     9
```

```
>> x*y
```

```
Error using *
Inner matrix dimensions
must agree.
```

```
>> x*y'
```

```
ans =
     6
```

```
>> x.*y
```

```
ans =
     1     2     3
```

```
>> B^2
```

```
ans =
    30    36    42
    66    81    96
   102   126   150
```

```
>> B.^2
```

```
ans =
     1     4     9
    16    25    36
    49    64    81
```

Otros tipos de datos

Estructuras (structs)

```
>> m
m =
  struct with fields:

    solver: 'ode45'
  extdata: [1×1 struct]
         x: [1×239 double]
         y: [4×239 double]
        xe: [1 7.5635 14.4492]
        ye: [4×3 double]
        ie: [1 2 3]
```

Acceso

```
>> m.ie
ans =
     1     2     3
```

Añadir campos

```
>> m.u=0
```

Celdas (cell array)

```
>> M{1}=m;M{2}=a
M =
  1×2 cell array
   {1×1 struct}   {1×126 double}
```

Almacenan datos de distinto tipo

Estructuras condicionales

Bucle for

```
>> phi0=linspace(0.1,25,20);  
>> for p=1:length(phi0)  
>>     disp(phi0(p));  
>> end
```

0.1000

1.4105

Permite iterar

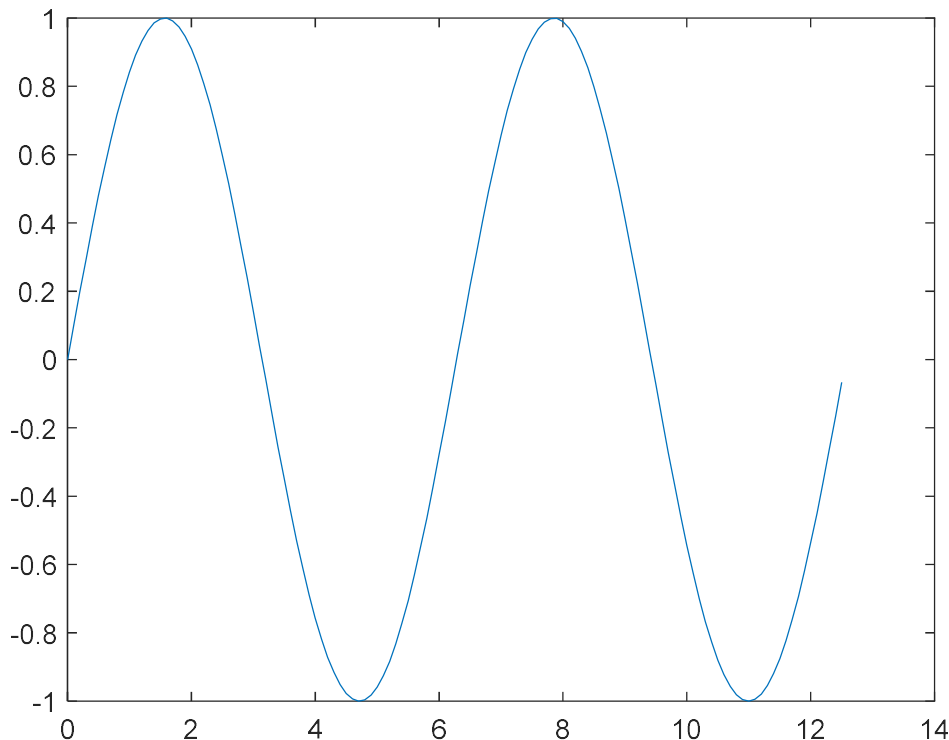
Clausula if

```
>> if (condición)  
>> ...  
>> else  
>> ...  
>> end
```

Permite ejecutar
condicionalmente

Gráficos básicos (plot)

```
>> a = 0:0.1:4*pi
>> figure('Windowstyle','docked');
>> plot(a,sin(a),'b')
```



plot

2-D line plot

Syntax

plot(X,Y)

plot(X,Y,LineSpec)

plot(X1,Y1,...,Xn,Yn)

plot(X1,Y1,LineSpec1,...,Xn,Yn,LineSpecn)

plot(Y)

Line Style	Description
-	Solid line (default)
--	Dashed line
:	Dotted line
-.	Dash-dot line
Marker	Description
o	Circle
+	Plus sign
*	
Color	Description
r	red
g	green
b	blue

Funciones

Inline functions

```
>> F=@(A,x) A*cos(x);  
>> F(1,pi)  
ans =  
    -1
```

Function file (.m)

drawCircle.m

```
function drawCircle()  
theta=linspace(0,2*pi,100);  
plot(cos(theta),sin(theta),'k');  
axis equal;
```

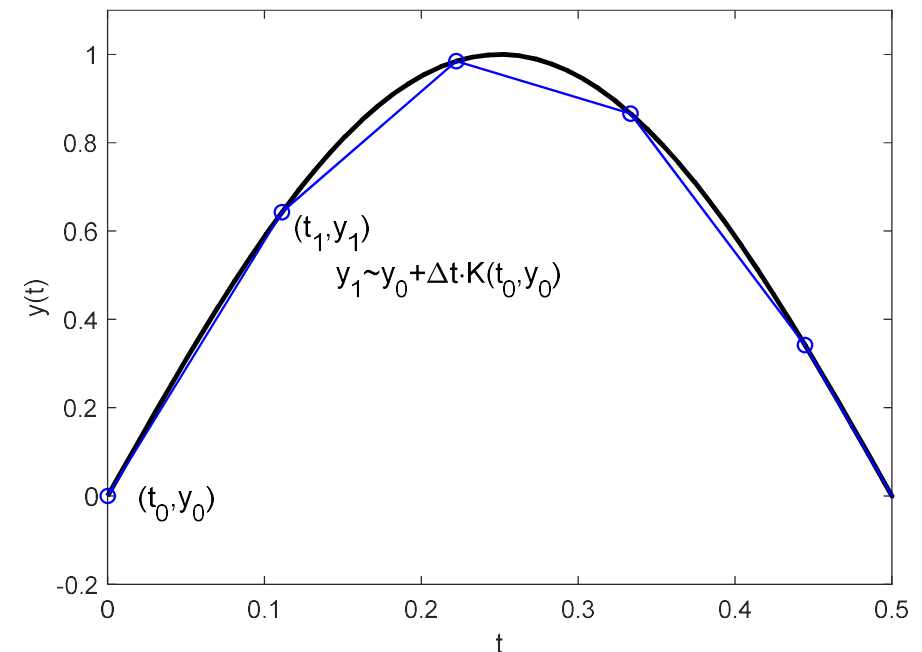
¿Qué es un integrador?

Un integrador es una rutina que proporciona una aproximación a la solución del problema de valor inicial

$$\dot{y}(t) = f(t, y)$$

$$y(t_0) = y_0$$

Para la solución numérica, se realiza una discretización del tiempo. Los distintos valores de $y_n \equiv y(t_n)$ son calculados de manera iterativa mediante un esquema.



El solver/integrador ode45

Matlab dispone de varios integradores para la resolución numérica de ecuaciones diferenciales. ode45 implementa un Runge-kutta de 4º orden con paso variable. Los distintos valores de y_n son calculados de manera iterativa:

$$\left. \begin{aligned} k_1 &= f(t_n, y_n) \\ k_2 &= f\left(t_n + \frac{\Delta t}{2}, y_n + k_1 \frac{\Delta t}{2}\right) \\ k_3 &= f\left(t_n + \frac{\Delta t}{2}, y_n + k_2 \frac{\Delta t}{2}\right) \\ k_4 &= f(t_n + \Delta t, y_n + k_3 \Delta t) \end{aligned} \right\} \rightarrow y_{n+1} = y_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

- El método es de orden 4, lo cual quiere decir que el error es de orden $O(\Delta t)^5$
- El paso en ode45 es adaptativo, es decir, se realizan integraciones adicionales para decidir qué paso es necesario hasta obtener un error inferior al impuesto por los parámetros '**RelTol**' y '**AbsTol**'.

El solver ode45

ode45

Solve nonstiff differential equations – medium order method

Syntax

```
[t,y] = ode45(odefun,tspan,y0)
[t,y] = ode45(odefun,tspan,y0,options)
[t,y,te,ye,ie] = ode45(odefun,tspan,y0,options)
sol = ode45(____)
```

odefun – sistema de ecuaciones a resolver

tspan – tiempos inicial y final

y0 – condición inicial para la integración

options – opciones del solver

t, y – vector de tiempos y soluciones

te, ye, ie – elementos relacionados con los eventos ('Events')

sol – estructura con todos los elementos de la solución

El solver ode45: argumentos

odefun	Función $f(t, y)$ de la ecuación. Dicha función puede invocarse desde un archivo .m, de la forma <pre>function yp = odefun(t, y) yp=[y(2); -y(1)];</pre> o bien, puede ser especificada como función inline en la forma <pre>sol=ode45(@(t, y) [y(2); -y(1)], ...</pre> sin la necesidad de crear un fichero adicional
tspan	Vector de tiempos. ode45 funciona con paso adaptativo, de manera que si se especifica $[t_0 \ t_1]$, el vector de tiempo de la solución no será equiespaciado. Se puede imponer que la solución sea proporcionada en puntos concretos de tiempo especificando aquí un vector (de más de dos elementos)
y0	Condiciones iniciales. Vector columna con tantos elementos como variables en la ecuación
options	Permite modificar parámetros del solver mediante la función odeset <pre><u>options</u> = odeset(<u>Name, Value, ...</u>)</pre>

El solver ode45: opciones

Los parámetros con los que más trabajaremos serán

'RelTol' — Relative error tolerance 1e-3 (default) 'AbsTol' — Absolute error tolerance 1e-6 (default)	En cada paso el solver estima el error e en la componente i de la solución. Para que el paso sea aceptado debe tener un error aceptable. El criterio implica a la tolerancia absoluta y relativa: $ e(i) \leq \max(\text{RelTol} * \text{abs}(y(i)), \text{AbsTol}(i))$
'MaxStep' —	Maximo paso de tiempo permitido en la solución
'Events' —	Función de eventos. Permite interceptar eventos concretos durante la integración. Se explicará más adelante

Para una lista más completa, consultar la documentación de **odeset**.

Eventos

La opción '**Events**' del integrador nos permite especificar el comportamiento del solver cuando se producen ciertas condiciones durante la integración.

Una función de eventos tiene que tener una estructura de tipo

```
function [position, isterminal, direction] = EventsFcn(t, y)
```

position	Define la variable que tiene que ser nula en el evento (por ejemplo, position=y(2) sirve para detectar el evento y(2)=0)
isterminal	Determina si la integración se detiene (1) o no (0) en el evento
direction	Determina si el evento se detecta cuando el valor de la condición es ascendente (1), descendente (-1), o se detecta en ambos casos (0)

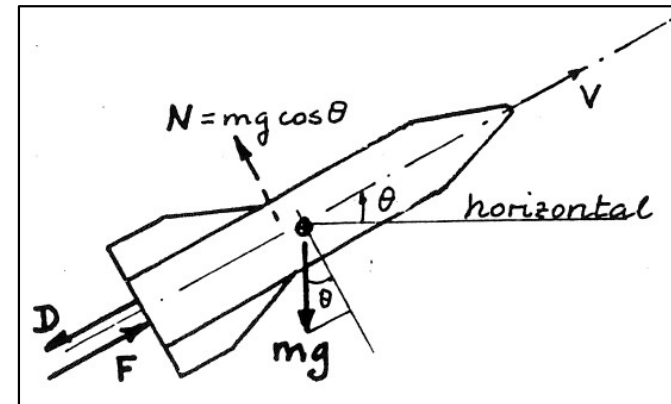
En el caso de cálculo de trayectorias, se puede detectar el apogeo ($v = y(1) = 0$, punto más alto, descendente) y el impacto ($z = y(2) = 0$, descendente). Se suele incluir todo en una función y separar también la definición de **odefun**, anticipando casos más complicados.

Vehículo propulsado (1D)

- Balance de fuerzas en la dirección de la velocidad. Ángulo de ataque nulo ($\alpha = 0$), ascensión vertical ($\theta = \pi/2$).

$$m \frac{dV}{dt} = F - D - m \cdot g$$

$$\frac{dz}{dt} = V$$



- Hipótesis:

- $F \approx -g_0 I_{sp} \dot{m} \cdot (t < t_b)$
- $D = \frac{1}{2} \rho(z) V^2 C_{D0} S_{ref}$

$$\dot{m} = -m_p^0 / t_b \rightarrow m = m^0 - m_p^0 t / t_b$$

(Combustible consumido a ritmo constante)

Ecuaciones unidimensionales

$$\frac{dV}{dt} = g_0 I_{sp} \frac{\varepsilon / t_b}{1 - \varepsilon t / t_b} (t < t_b) - g(z) - \frac{1}{2} \frac{\rho(z) C_{D_0} S_{ref}}{m_0} \frac{V |V|}{1 - \varepsilon \min(t, t_b) / t_b}$$

$$\frac{dz}{dt} = V$$

$$\varepsilon = m_p^0 / m^0$$

$$\min(t, t_b) = \begin{cases} t & t < t_b \\ t_b & t \geq t_b \end{cases}$$

1-Lanzamiento vertical y sin resistencia aerodinámica

$$g = g_0, C_D = 0$$

$$\begin{aligned} \frac{dv}{dt} &= g_0 I_{sp} \frac{\varepsilon / t_b}{1 - \varepsilon t / t_b} (t < t_b) - g_0 \\ \frac{dz}{dt} &= v \end{aligned} \quad \rightarrow \quad \frac{d}{dt} \begin{bmatrix} v \\ z \end{bmatrix} \equiv \begin{bmatrix} \dot{y}_1 \\ \dot{y}_2 \end{bmatrix} = \begin{bmatrix} g_0 I_{sp} \frac{\varepsilon / t_b}{1 - \varepsilon t / t_b} \cdot (t < t_b) - g_0 \\ y_1 \end{bmatrix}$$

Resolver el caso sencillo añadiendo la condición de parada ($z = 0$), y recoger también los eventos de **burnout** ($t = t_b$) y **apogeo** ($v = 0$), Observar la dependencia de las soluciones con t_b .
Fijar $\varepsilon = 0.9, I_{sp} = 250 \text{ s}$.

Aplicación-1: Ecuaciones y parámetros

```
params.g0 = 9.81; % [m/s]
params.lsp = 250; % [s]
params.eps = 0.9; % [1]
params.tb = 25; % [s]
params.Sref = 0.1; % [m^2]
params.Cd = 0.8; % [1]
params.m0 = 1000; % [kg]
params.g = 1; % [1]
params.Rt = 6370e3; % [m]

options = odeset('MaxStep',.5,'AbsTol',1e-4,'RelTol', 1e-4, 'Events', @(t, Y) EventsFcn(t, Y, params));

m = ode45(@(t, Y) misil_1D(t, Y, params), [0 10000], Y0, options);

function dY_dt = misil_1D(t, Y, p)
dY_dt(1,1) = p.g0*p.lsp*p.eps/p.tb/(1-p.eps*t/p.tb)*(t<p.tb)...
-p.g0);
dY_dt(2,1) = Y(2);
end

function [position,isterminal,direction] = EventsFcn(t,y,p)
position=[p.tb-t;y(1)];
isterminal=[0;1];
direction=[-1;-1];
end
```

Aplicación-1: estructura resultados

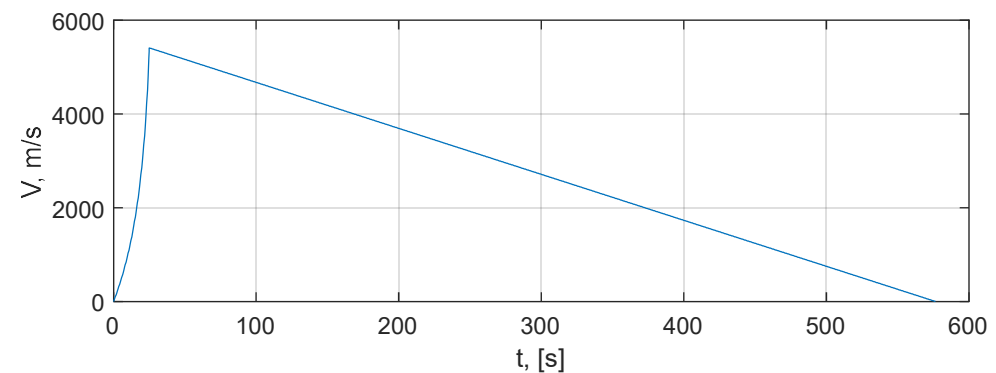
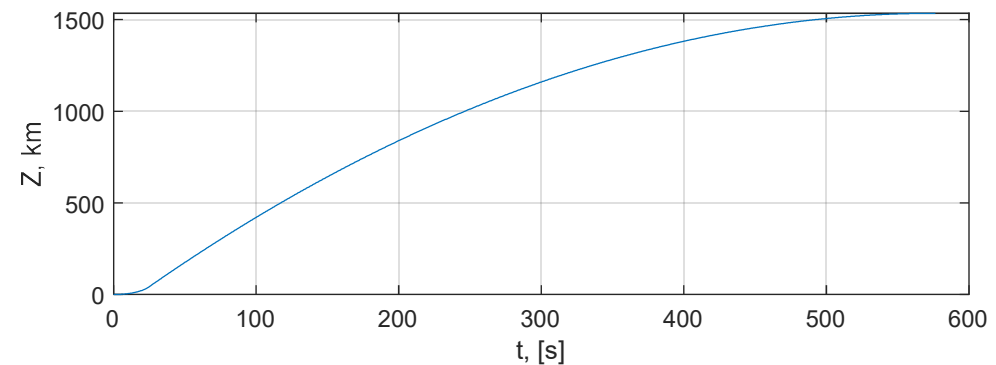
`m = ode45(@(t, Y) misil_1D(t, Y, params), [0 10000], Y0, options)`
`m = struct with fields:`

```

solver: 'ode45'
extdata: [1x1 struct]
  x: [0 0.0016 0.0097 0.0501 0.2520 0.7520 1.2520 1.7520 2.2520 2.7520
3.2520 3.7520 4.2520 4.7520 ... ]
  y: [2x1160 double]
  xe: [25 576.5458]
  ye: [2x2 double]
  ie: [1 2]
stats: [1x1 struct]
idata: [1x1 struct]

```

La estructura de solución contiene la detección del fin de combustión en $t=25$ s y para la simulación en el apogeo



Aplicación -1: Cambio de eventos

```
function [position, isterminal, direction] = EventsFcn(t,y,p)
position=[y(1); y(2); p.tb-t];
isterminal=[0; 1;0];
direction=[-1; -1; -1];
end
```

struct with fields:

solver: 'ode45'

extdata: [1 × 1 struct]

x: [0 0.0016 0.0097 0.0501 0.2520 0.7520 1.2520 1.7520 2.2520 2.7520 3.2520 3.7520 4.2520 4.7520 ...]

y: [2 × 2279 double]

xe: [25 576.5458 1.1359e+03]

ye: [2 × 3 double]

ie: [3 1 2]

stats: [1 × 1 struct]

idata: [1 × 1 struct]

Eventos: apogeo, impacto,
fin de combustión

Aplicación -1: Representación Resultados

```
m = ode45(@(t, Y) misil_1D(t, Y, params), [0 20000], Y0, options);
```

```
time=m.x;
```

```
z=m.y(2,:);
```

```
v=m.y(1,:);
```

```
figure(1)
```

```
subplot(2,1,1)
```

```
plot(time,z)
```

```
grid;xlabel('t, [s]');
```

```
ylabel('Z, m')
```

```
hold on
```

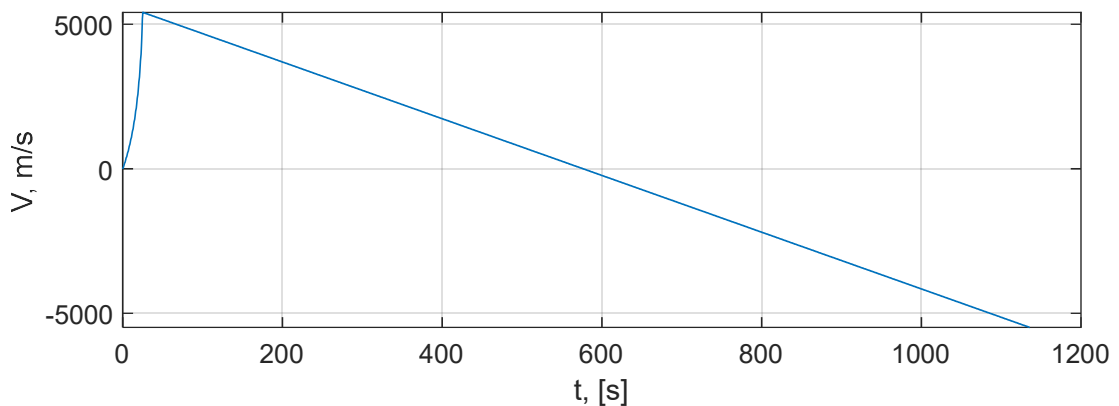
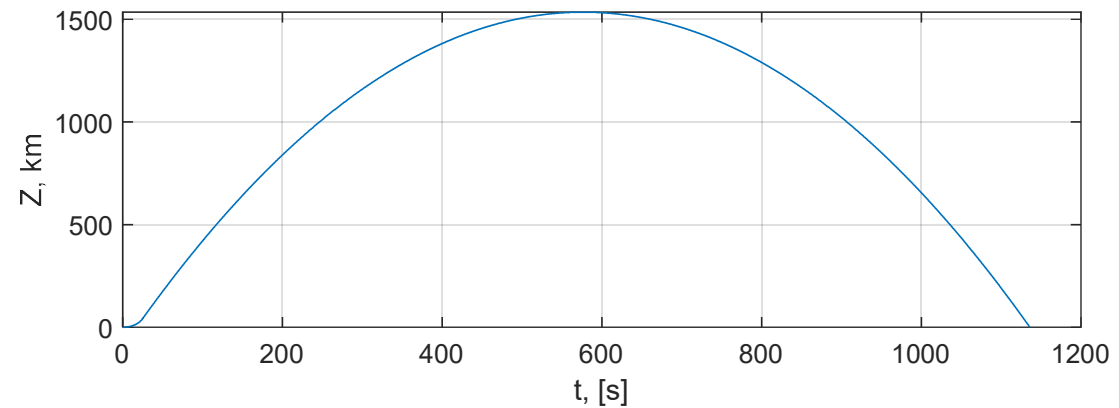
```
subplot(2,1,2)
```

```
plot(time,v)
```

```
grid;xlabel('t, [s]');
```

```
ylabel('V, m/s')
```

```
hold on
```



2-Variación de la gravedad

Añadiendo la variación de
la gravedad con la altura

```
function dY_dt = misil_1D(t, Y, p)
dY_dt(1,1) = p.g0*p.lsp*p.eps/p.tb/(1-p.eps*t/p.tb)*(t<p.tb)...
- gravity(Y(2),p);
dY_dt(2,1) = Y(1);
end
```

```
function g = gravity(z, p)
g = p.g0/(1+z/p.Rt)^2;
end
```

Comparación con el caso anterior

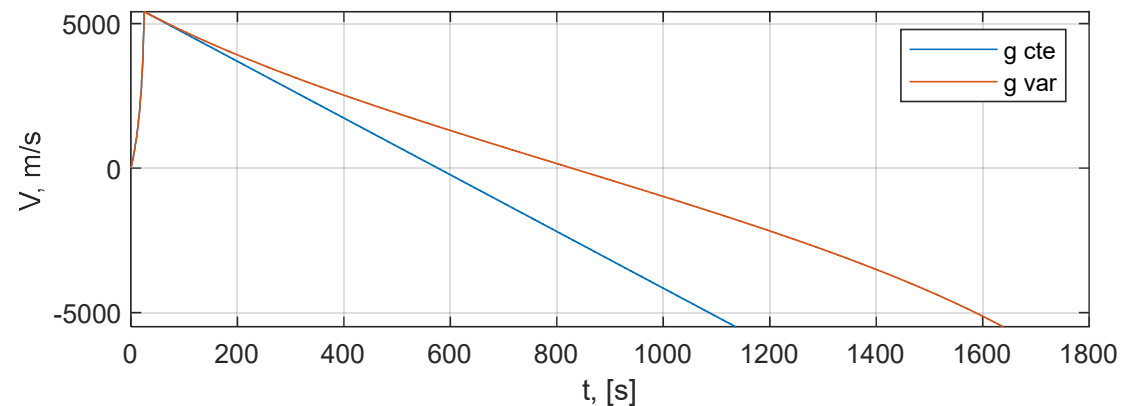
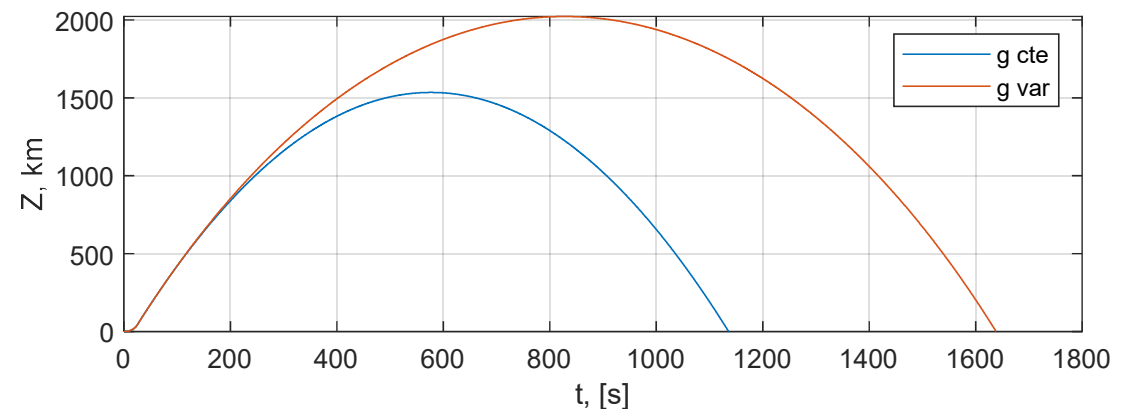
Usar `hold on` para no borrar las curvas
anteriores al añadir una nueva en la figura.

Usar `legend` para identificar las curvas

```
legend('g cte','g var')
```

$$\frac{dv}{dt} = \frac{\varepsilon}{1 - \varepsilon t / t_b} \cdot (t < t_b) - g_0 \frac{1}{(1 + z / R_T)^2}$$

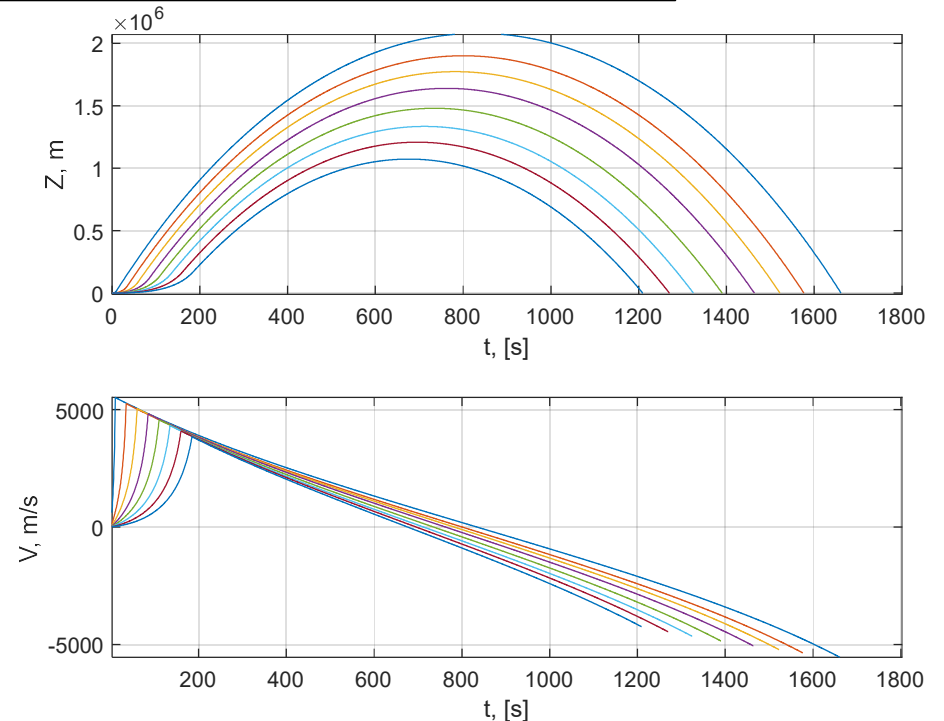
$$\frac{dz}{dt} = v$$



Aplicación -2: Barrido de parámetros

```
tb = [10:25:200];
for ii = 1:length(tb)
    params.tb = tb(ii);
    options = odeset('MaxStep',.5,'AbsTol',1e-4,'RelTol', 1e-4, 'Events', @(t, Y) EventsFcn(t, Y, params));
    Y0=[0;0]; %[m; m/s];
    m = ode45(@(t, Y) misil_1D(t, Y, params), [0 20000], Y0, options);
    h_b(ii)=m.ye(2,1);
    v_b(ii)=m.ye(1,1);
    hmax(ii)=m.ye(2,2);
end
```

Barrido en 1 parámetro, (t_b)



Aplicación -2: Comparación con cálculos analíticos

burnout

$$V_b = g_0 \cdot I_{sp} \cdot \left(F_1(\epsilon) - \frac{t_b}{I_{sp}} \right)$$

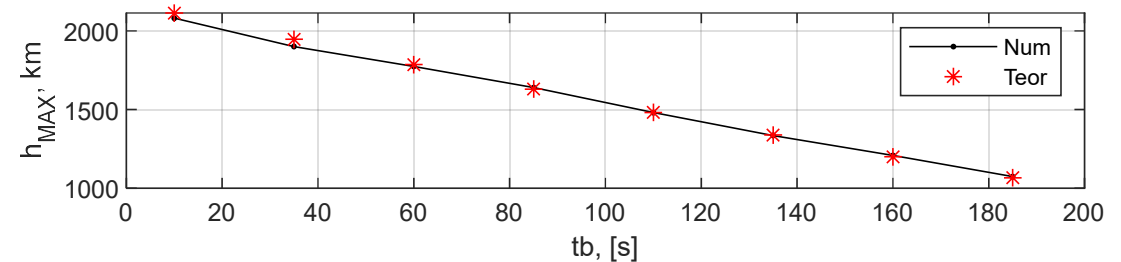
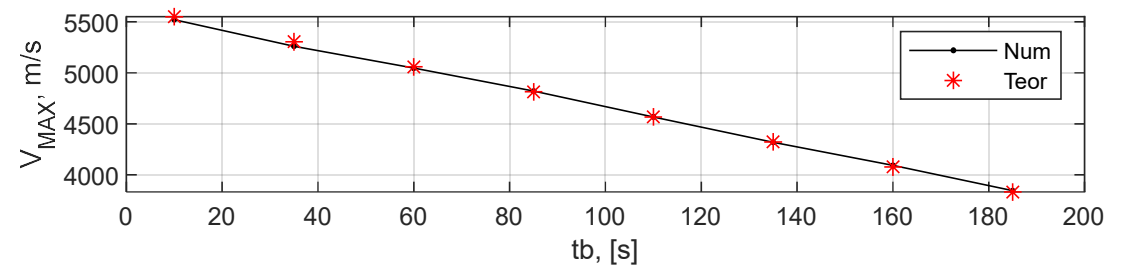
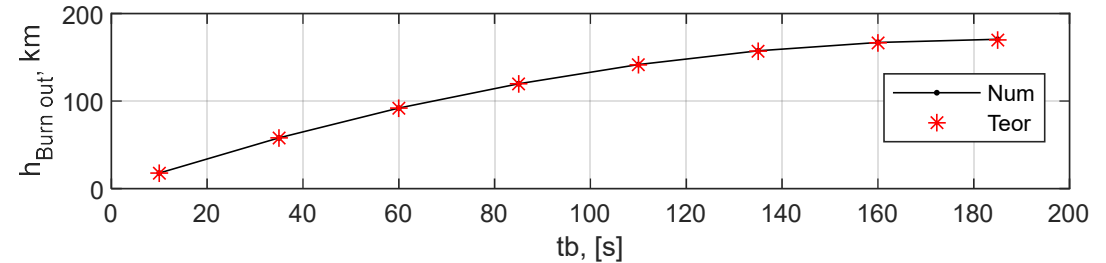
$$h_b = g_0 \cdot I_{sp} \cdot t_b \left(F_2(\epsilon) - \frac{1}{2} \cdot \frac{t_b}{I_{sp}} \right)$$

$$F_1(\epsilon) = -\log(1 - \epsilon)$$

$$F_2(\epsilon) = \frac{\epsilon + (1 - \epsilon)\ln(1 - \epsilon)}{\epsilon}$$

apogeo

$$h_c = \frac{V_b^2}{2g_0} \cdot \frac{\left(1 + \frac{h_b}{R_{Earth}} \right)^2}{1 - \frac{V_b^2 \cdot \left(1 + \frac{h_b}{R_{Earth}} \right)}{2g_0 R_{Earth}}}$$



3-Rozamiento aerodinámico

$$\frac{dv}{dt} = \frac{\varepsilon}{1 - \varepsilon t / t_b} \cdot (t < t_b) - g_0 \frac{1}{(1 + z / R_T)^2} - \frac{1}{2} \frac{\rho(z) C_{D_0} S_{ref}}{m_0} \frac{v |v|}{(1 - \varepsilon \min(t, t_b) / t_b)}$$

$$\frac{dz}{dt} = v$$

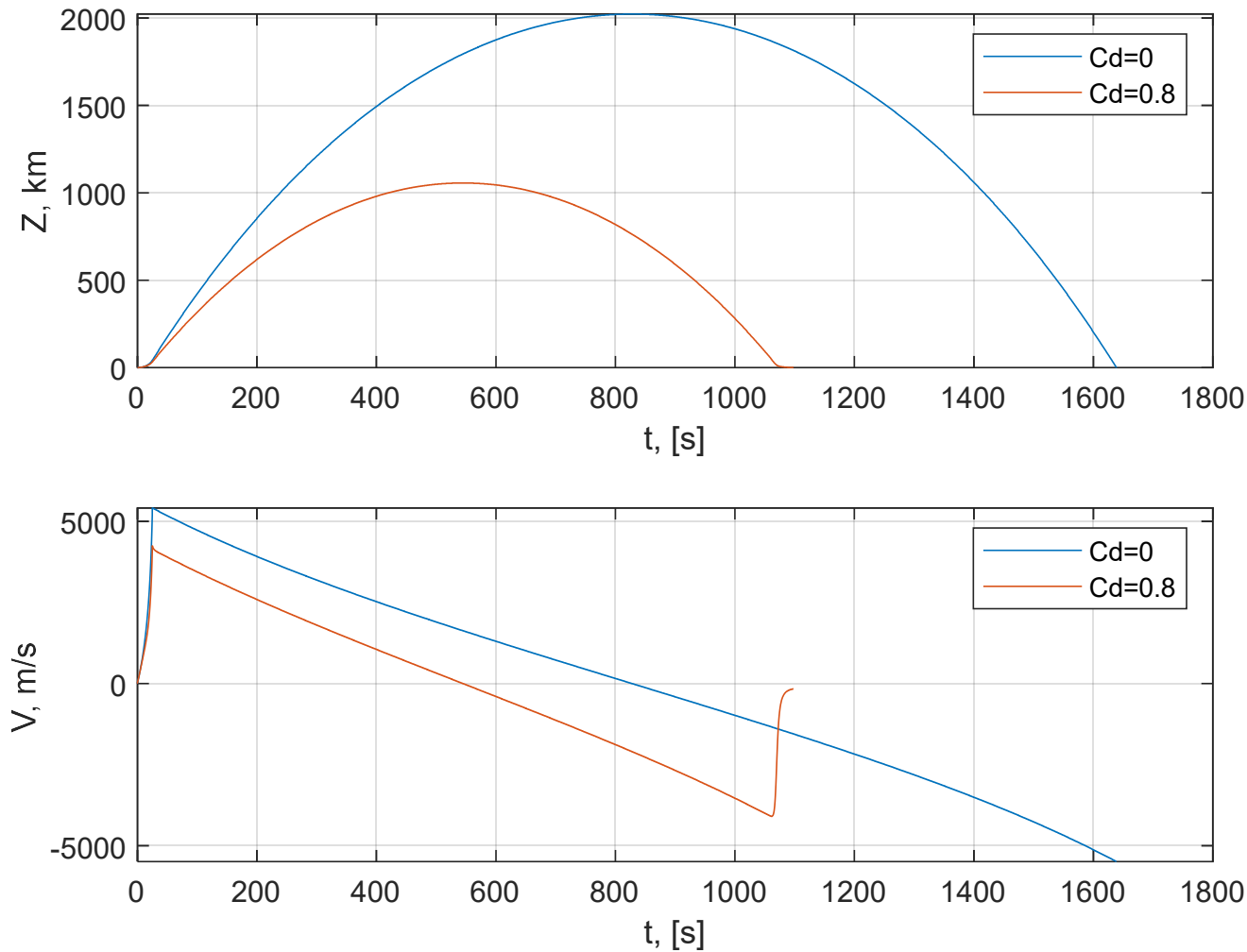
Fdrag

Para incluir los efectos aerodinámicos se necesitarán, además de los detalles del propulsante (m_p^0 , I_{sp} , t_b), parámetros de diseño (S_{ref} , m_0 , C_{D_0}), un modelo de la variación de la densidad con la altitud. Se escogerá el denominado modelo de atmósfera standard internacional (ISA, ver apéndice 1).

```
function dY_dt = misil_1D(t, Y, p)
dY_dt(1,1) = Y(2);
dY_dt(2,1) = p.g0*p.lsp*p.eps/p.tb/(1-p.eps*t/p.tb)*(t<p.tb)...
- gravity(Y(1),p)-Fdrag(t,Y,p);
end

function Fd=Fdrag(t,y,p)
Fd=.5*p.Sref*p.Cd/p.m0*...
rhoISA(y(1,:)).*y(2,:).*abs(y(2))./(1-p.eps*min(t,p.tb)/p.tb);
end
```

Aplicación -3: Efecto de la resistencia aerodinámica



Apéndice 1 - International Standard Atmosphere (ISA)

Valores de referencia, nivel de mar standard, $r = R_T = 6378 \text{ km}$

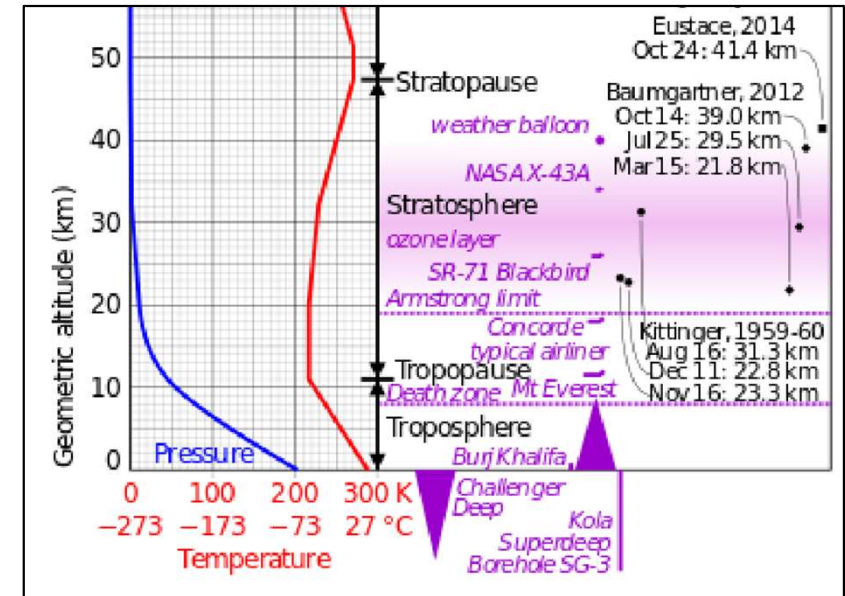
$p_0 = 101.32 \text{ kPa} = 1.013 \text{ bar}$	$c_{p0} = 1005 \text{ J kg}^{-1} \text{ K}^{-1}$
$\rho_0 = 1.225 \text{ kg m}^{-3}$	$c_{v0} = 717.98 \text{ J kg}^{-1} \text{ K}^{-1}$
$T_0 = 15^\circ \text{ C} (288.2 \text{ K})$	
$c_{s0} = 340.3 \text{ m s}^{-1}$	
$g_0 = 9.8066 \text{ m s}^{-2}$	

Variaciones con la altitud

$$\boxed{\begin{aligned} p &= \rho RT(h) \\ \frac{dp}{dh} &= -\rho g \end{aligned}}$$



$$\begin{cases} \frac{dp}{p} = -\frac{g}{RT(h)} dh \\ \rho = \frac{1}{RT(h)} p(h) \\ c_s = \sqrt{\gamma RT(h)} \end{cases}$$



Troposfera	$T(h) = T_0(1 - h/h_1)$	$p = p_0 (1 - h/h_1)^{\frac{g_0 h_1}{RT_0}}$	$\rho = \rho_0 (1 - h/h_1)^{\frac{g_0 h_1}{RT_0} - 1}$
	$h_1 = 44.34 \text{ km}, g_0 h_1 / RT_0 = 5.25$		
Estratosfera	$T(h) = T_s = 216.7 \text{ K}$	$p = p_s e^{-\frac{(h-h_s)}{H_2}}$	$\rho = \rho_s e^{-\frac{(h-h_s)}{H_2}}$
	$h_s = 11 \text{ km}, \rho_s = 0.365 \text{ kg/m}^3, p_s = 0.223 \text{ bar}, H_2 = RT_s / g_0 = 6.35 \text{ km}$		